



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

M.C. René Adrián Dávila Pérez

Asignatura:

Programación orientada a objetos

Grupo:

07

No de práctica(s):

REPORTE DE PROYECTO: ETAPA 1

Integrante(s):

424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada:

1

Semestre:

2027 - 1

Fecha de entrega:

28 de septiembre de 2026

Observaciones:

Calificación:

Índice

1. INTRODUCCIÓN	2
1.1. Objetivo General	2
1.2. Objetivos Específicos	2
2. MARCO TEÓRICO	2
2.1. Paradigma Orientado a Objetos y Encapsulamiento	2
2.2. Sobrecarga de Métodos (Overloading)	3
2.3. Máquinas de Estado y Control de Ciclo de Vida	3
2.4. Gestión de Memoria mediante Arreglos Estáticos	3
2.5. Algoritmo de Ordenamiento por Intercambio y Comparación de Cadenas	3
3. DESARROLLO Y ARQUITECTURA DEL SISTEMA	4
3.1. Descripción Algorítmica de las Clases	4
3.1.1. Clase Sistema	4
3.1.2. Clase Medico	5
3.1.3. Clase Enfermero	5
3.1.4. Clase Paciente	6
3.1.5. Clase Main	6
4. RESULTADOS Y ANÁLISIS DE EJECUCIÓN	6
4.1. Capturas de Pantalla de la Ejecución del Simulador	7
4.2. Análisis Detallado del Escenario de Prueba	7
5. CONCLUSIONES	8
6. REFERENCIAS	8

1. INTRODUCCIÓN

En la actualidad, el desarrollo de sistemas computacionales aplicados a la gestión administrativa y médica en instituciones de salud resulta fundamental para garantizar la eficiencia y la calidad en la atención de los usuarios. Un sistema hospitalario requiere controlar minuciosamente el flujo de pacientes, la asignación de personal capacitado según la especialidad correspondiente y el seguimiento continuo del estado de salud de cada individuo.

En el paradigma de la Programación Orientada a Objetos, este tipo de problemáticas se modela mapeando los entes del mundo real, tales como médicos, enfermeros, pacientes y el propio hospital hacia entidades digitales compuestas por atributos y comportamientos interrelacionados.

1.1 Objetivo General

Crear un sistema en Java que simule el control y la atención dentro de un hospital, permitiendo registrar y gestionar el trabajo de médicos, enfermeros y la atención a pacientes, así como controlar citas, consultas y tratamientos de forma ordenada.

1.2 Objetivos Específicos

- Modelar e implementar las clases principales del dominio hospitalario (Sistema, Médico, Enfermero, Paciente y Main) aplicando principios de encapsulamiento mediante modificadores de acceso privados y métodos de acceso y modificación.
- Implementar el mecanismo de sobrecarga de métodos (method overloading) en la clase Sistema para adaptar la asignación de pacientes según se trate de personal médico o de enfermería.
- Gestionar las reglas de negocio del hospital mediante estructuras continuas de memoria (arreglos estáticos), respetando los límites de capacidad por profesional (máximo 10 pacientes por médico y 3 por enfermero) y validando la coincidencia estricta de especialidades.
- Controlar la máquina de estados del paciente (PENDIENTE, EN CONSULTA, EN TRATAMIENTO) y garantizar que las consultas médicas se realicen de manera individual e ininterrumpida.
- Implementar el algoritmo de ordenamiento por intercambio para generar reportes alfabéticos descendentes de las listas de pacientes sin alterar el orden original de almacenamiento en memoria.
- Diseñar una interfaz interactiva en consola en la clase Main mediante un menú estructurado con lecturas seguras de datos de entrada.

2. MARCO TEÓRICO

El desarrollo de software estructurado bajo el paradigma de la Programación Orientada a Objetos exige fundamentar el diseño en conceptos teóricos precisos que rigen la arquitectura del código y la gestión de memoria durante la ejecución.

2.1 Paradigma Orientado a Objetos y Encapsulamiento

El encapsulamiento es un principio fundamental de la programación orientada a objetos enfocado en ocultar los detalles internos de la implementación de una clase y proteger el estado directo de los objetos ante modificaciones no controladas desde el exterior [1]. Este mecanismo agrupa atributos y comportamientos en una misma entidad, restringiendo el acceso

mediante modificadores de visibilidad como `private` [1]. La lectura y actualización de los datos se gestiona exclusivamente a través de métodos de acceso públicos (getters y setters), lo que permite integrar filtros de validación que aseguran la consistencia y la integridad de los datos en todo momento [1].

2.2 Sobrecarga de Métodos (Overloading)

La sobrecarga de métodos constituye una forma de polimorfismo estático que permite declarar múltiples métodos dentro de una misma clase compartiendo el mismo identificador, siempre que sus firmas sean claramente distintas [2, 3]. La diferenciación de las firmas se establece mediante la cantidad de parámetros, sus tipos de datos o el orden en que son definidos dentro de la declaración [2, 3]. La resolución sobre qué método ejecutar se lleva a cabo durante la fase de compilación (static binding), lo que facilita el diseño de interfaces limpias y uniformes para ejecutar operaciones afines sobre diferentes tipos o volúmenes de datos [2].

2.3 Máquinas de Estado y Control de Ciclo de Vida

El estado de un objeto representa la condición o situación particular en la que se encuentra en un instante determinado dentro de su ciclo de vida, la cual está definida por los valores almacenados en sus atributos [4, 5]. El modelo de máquina de estados define el comportamiento dinámico de una entidad estableciendo un conjunto de estados posibles y las transiciones válidas entre ellos, las cuales son desencadenadas por la invocación de métodos específicos [5]. Este control garantiza que la entidad solo ejecute operaciones coherentes con su estado actual, evitando inconsistencias dentro de la lógica del sistema.

2.4 Gestión de Memoria mediante Arreglos Estáticos

En la máquina virtual de Java, los arreglos son objetos estructurados que almacenan colecciones homogéneas de elementos en posiciones contiguas dentro de la memoria Heap [6]. Al tratarse de estructuras estáticas, su tamaño se define de forma fija e inmutable desde el momento de la instanciación [6]. La administración eficiente de arreglos estáticos requiere implementar contadores enteros auxiliares que permitan llevar el control del índice de ocupación, evitando excepciones por desbordamiento de límites (`ArrayIndexOutOfBoundsException`) y optimizando las búsquedas al iterar únicamente sobre las posiciones que contienen datos reales [6].

2.5 Algoritmo de Ordenamiento por Intercambio y Comparación de Cadenas

La evaluación lexicográfica de texto en Java se realiza mediante el método `compareToIgnoreCase()` de la clase `String`, el cual compara dos cadenas de texto de forma alfabética omitiendo diferencias entre mayúsculas y minúsculas [7]. El método retorna un valor entero negativo, cero o positivo según el orden relativo Unicode de los caracteres evaluados [7]. Este valor es indispensable para implementar algoritmos de ordenamiento por intercambio (Burbuja), donde se analizan elementos adyacentes de un arreglo y se efectúan intercambios de posición con apoyo de variables auxiliares para organizar las listas en el sentido deseado [7].

3. DESARROLLO Y ARQUITECTURA DEL SISTEMA

Para dar cumplimiento a los requerimientos planteados para la etapa 1 del simulador hospitalario, diseñamos e implementamos una arquitectura modular compuesta por 5 clases principales. A continuación se detalla la estructura lógica de cada componente.

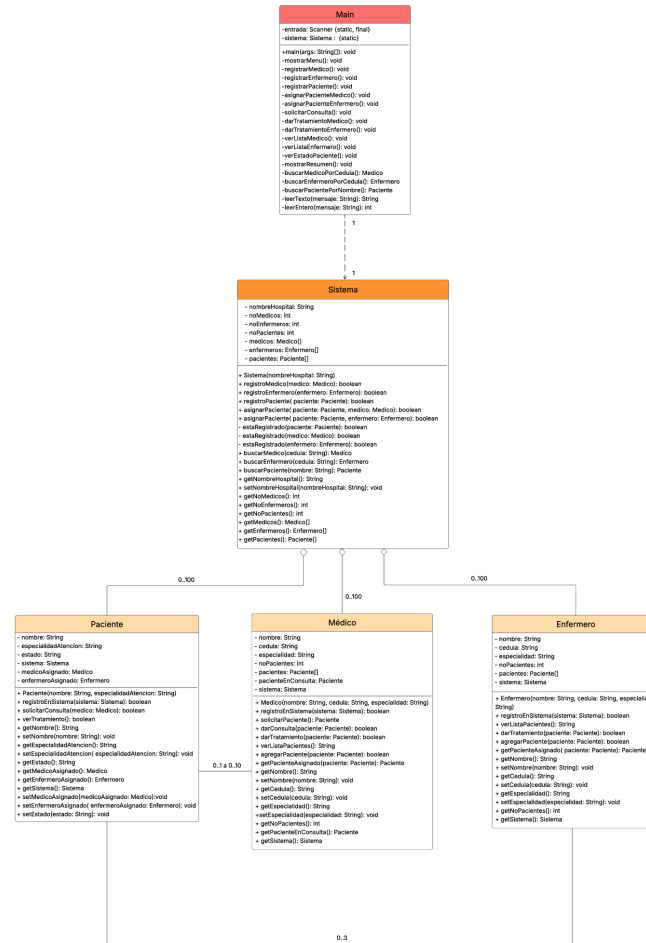


Figura 1. Diagrama de clases UML completo del simulador hospitalario (Etapa 1), representando la arquitectura modular del sistema, el encapsulamiento mediante modificadores de acceso privados y las relaciones de asociación entre Main, Sistema, Medico, Enfermero y Paciente.

3.1 Descripción Algorítmica de las Clases

3.1.1 Clase Sistema

La clase Sistema actúa como el controlador central del hospital. Contiene atributos privados para almacenar la razón social del hospital (nombreHospital), contadores numéricos para el registro activo (noMedicos, noEnfermeros, noPacientes) y arreglos unidimensionales estáticos con una capacidad máxima fijada en 100 posiciones para cada entidad (medicos, enfermeros, pacientes).

- **Registros de Personal y Pacientes (registroMedico, registroEnfermero, registroPaciente):** Evalúan que el objeto recibido no sea nulo, que la estructura de almacenamiento correspondiente disponga de espacio (contador menor a 100) y que no exista un registro duplicado en la memoria (verificando cédula para personal o nombre para pacientes). Si las condiciones se cumplen, almacenan la referencia en la posición del contador actual e incrementan el contador en una unidad.
- **Sobrecarga del Método asignarPaciente:** Se implementaron dos firmas para el método asignarPaciente:

-
1. **asignarPaciente(Paciente paciente, Medico medico):** Valida la existencia física y el registro activo de ambas entidades en el sistema. Posteriormente, delega la operación al método **agregarPaciente** del objeto Medico.
 2. **asignarPaciente(Paciente paciente, Enfermero enfermero):** Valida la existencia de ambos objetos y delega la asignación al método **agregarPaciente** de la clase Enfermero.
- **Búsquedas y Verificaciones Privadas:** Incluye métodos de búsqueda (**buscarMedico**, **buscarEnfermero**, **buscarPaciente**) que recorren linealmente los arreglos comparando cadenas mediante **equals** o **equalsIgnoreCase**. Los métodos privados **estaRegistrado** verifican la coincidencia de referencias de memoria en las listas del sistema.

3.1.2 Clase Medico

Representa al profesional de la medicina dentro del simulador. Posee atributos privados de identificación (nombre, cedula, especialidad), un contador de pacientes vinculados (**noPacientes**), un arreglo de referencias a **Paciente** con tamaño estático de 10 posiciones, un atributo para controlar el paciente actualmente atendido (**pacienteEnConsulta**) y una referencia al objeto **Sistema** al que pertenece.

- **Método agregarPaciente:** Verifica que el paciente no sea nulo, que el contador **noPacientes** no haya alcanzado el límite estricto de 10, que la especialidad requerida por el paciente coincida exactamente con la del médico y que el paciente no haya sido asignado previamente a este mismo médico. Al cumplir con las reglas, guarda al paciente en el arreglo, incrementa el contador y actualiza el médico asignado en la entidad del paciente.
- **Método darConsulta:** Gestiona el inicio de la atención médica. Valida que el paciente esté asignado al médico, que coincidan las especialidades, que el paciente se encuentre en estado “PENDIENTE” y que el médico no tenga a otro paciente en consulta en ese momento (**pacienteEnConsulta == null**). Al autorizar la consulta, asigna la referencia a **pacienteEnConsulta** y actualiza el estado del paciente a “EN CONSULTA”.
- **Método darTratamiento:** Cambia el estado del paciente asignado a “EN TRATAMIENTO”. En caso de que dicho paciente fuera el que se encontraba en consulta activa, libera la variable **pacienteEnConsulta** (asignándole valor nulo) para permitir la entrada de un nuevo paciente.
- **Método verListaPacientes:** Crea una copia auxiliar del arreglo de pacientes asignados hasta la posición **noPacientes**. Implementa un algoritmo de ordenamiento por intercambio (Burbuja) comparando los nombres mediante **compareToIgnoreCase()** para ordenar la copia en formato alfabético descendente (de la Z a la A). Retorna una cadena estructurada con los nombres y especialidades del personal ordenado.

3.1.3 Clase Enfermero

Representa al personal de enfermería encargados del seguimiento de los pacientes. Estructuralmente similar a la clase **Medico**, pero ajustada a sus propias reglas de negocio: posee un arreglo de pacientes asignados con un límite máximo estricto de 3 elementos.

- **Método agregarPaciente:** Valida que el paciente no sea nulo, que el enfermero no sobrepase el cupo máximo de 3 pacientes, la concordancia de la especialidad y la ausencia de duplicados en su lista. Al concretarse la asignación, almacena al paciente, incrementa el contador y vincula al enfermero en la entidad paciente.
- **Método darTratamiento:** Verifica la asignación previa del paciente y actualiza directamente su estado a “EN TRATAMIENTO”.

-
- **Método verListaPacientes:** Genera un arreglo copia de su lista activa y aplica el algoritmo de ordenamiento por intercambio en sentido alfabético descendente. Devuelve el reporte en texto plano respetando el principio de inmutabilidad de la lista original.

3.1.4 Clase Paciente

Modela a los usuarios del hospital. Contiene atributos privados para el nombre, la especialidad de atención requerida, el estado del paciente en el flujo hospitalario (estado), así como las referencias directas al Sistema, a su Medico asignado y a su Enfermero asignado.

- **Constructor:** Inicializa el nombre y la especialidad de atención, estableciendo por defecto el estado inicial del paciente en “PENDIENTE”.
- **Método solicitarConsulta:** Permite al paciente iniciar la interacción médica. Verifica que el médico indicado no sea nulo, que el estado actual del paciente sea “PENDIENTE”, que la especialidad coincida y que se encuentre registrado dentro de la lista de pacientes del médico. Transfiere la ejecución al método darConsulta del médico.
- **Método verTratamiento:** Evalúa el estado del paciente y retorna un valor booleano verdadero únicamente cuando el atributo estado es exactamente igual a la cadena “EN TRATAMIENTO”.

3.1.5 Clase Main

Actúa como la interfaz de usuario en consola y punto de entrada de la aplicación. Declara dos atributos de clase estáticos: un objeto de la clase Scanner para la lectura centralizada de datos y una instancia de la clase Sistema.

- **Control del Flujo (main):** Solicita el nombre del hospital al iniciar la ejecución. Despliega recursivamente un menú interactivo con 12 opciones principales más la opción de salida (0) utilizando un bucle do-while y una estructura de selección múltiple switch-case.
- **Métodos Auxiliares de Lectura Segura:** leerTexto procesa cadenas eliminando espacios en blanco innecesarios mediante .trim(). leerEntero implementa un ciclo infinito de captura asistido por un bloque de manejo de excepciones try-catch para capturar la excepción NumberFormatException, evitando el colapso de la consola ante entradas no numéricas.

4. RESULTADOS Y ANÁLISIS DE EJECUCIÓN

Para comprobar el correcto funcionamiento del simulador y garantizar el cumplimiento de las reglas de negocio establecidas para la etapa 1, se llevó a cabo una sesión integral de pruebas dentro de la interfaz en consola de la clase Main. Durante esta ejecución se validaron las 12 funcionalidades contempladas en el sistema, abarcando desde el registro y búsqueda de personal hasta el control del ciclo de vida de los pacientes y la emisión de reportes.

A continuación, se presenta y analiza la ejecución representativa correspondiente al flujo de atención de enfermería, la cual evidencia la interacción entre la lectura de datos, las búsquedas por clave en memoria y la actualización directa de la máquina de estados del paciente.

4.1 Capturas de Pantalla de la Ejecución del Simulador

```
=====
UNAM
=====
1. Registrar medico
2. Registrar enfermero
3. Registrar paciente
4. Asignar paciente a medico
5. Asignar paciente a enfermero
6. Solicitar consulta
7. Dar tratamiento a paciente (medico)
8. Dar tratamiento a paciente (enfermero)
9. Ver lista de pacientes de un medico
10. Ver lista de pacientes de un enfermero
11. Ver estado de un paciente
12. Mostrar resumen del sistema
0. Salir
=====
Seleccione una opcion: 8
--- Tratamiento por enfermero ---
Cedula del enfermero: 2
Apellidos y nombres del paciente: Lopez Tadeo
=====
Seleccione una opcion: 8
--- Tratamiento por enfermero ---
Cedula del enfermero: 2
Apellidos y nombres del paciente: Lopez Tadeo
El paciente esta ahora en tratamiento.
=====
```

Figura 2: Ejecución de la opción 8 del menú interactivo “Tratamiento por enfermero”, donde se captura la cédula del enfermero, se localiza al paciente “Lopez Tadeo” y se concreta la transición exitosa de su estado a “EN TRATAMIENTO”.

4.2 Análisis Detallado del Escenario de Prueba

A partir del caso de prueba mostrado en la Figura 2, se detalla el comportamiento interno y el flujo de ejecución de la arquitectura orientada a objetos:

1. **Captura y Control de Flujo (Main):** El usuario seleccionó la opción 8 en el menú principal desplegado mediante un ciclo do-while. La lectura entera del número se realizó con el método auxiliar leerEntero(), garantizando la captura segura de datos frente a posibles errores de tipeo en la consola.
2. **Búsqueda e Identificación de Entidades (Sistema):** Tras solicitar la cédula del enfermero (2), la clase Main invocó al método buscarEnfermero(“2”) perteneciente a la clase Sistema. Este realizó un recorrido lineal en el arreglo estático de enfermeros para localizar el objeto correspondiente y retornar su referencia.
3. **Validación de Reglas de Negocio (Enfermero):** Posteriormente, se solicitó la identificación del paciente (Lopez Tadeo). El objeto Enfermero verificó dentro de su arreglo interno que la referencia del paciente no fuera nula, que estuviera previamente asignado a su lista de atención y que existiera concordancia en la especialidad médica solicitada.
4. **Transición en la Máquina de Estados (Paciente):** Una vez validadas las condiciones anteriores, la entidad Enfermero ejecutó su método darTratamiento(), el cual modificó directamente el atributo privado estado del objeto Paciente, cambiando su valor de “PENDIENTE” o “EN CONSULTA” hacia “EN TRATAMIENTO”.
5. **Retroalimentación y Continuidad:** Finalmente, la consola imprimió el mensaje de confirmación “El paciente esta ahora en tratamiento.” y devolvió el control al menú

principal, demostrando la estabilidad del programa y la ausencia de excepciones no controladas durante la ejecución.

5. CONCLUSIONES

Desarrollar este simulador nos demostró que la Programación Orientada a Objetos va mucho más allá de escribir código funcional; se trata de saber modelar la complejidad del mundo real. Al llevar la operación de un hospital a un sistema digital, nos dimos cuenta de que la clave para que todo funcione bien no es amontonar lógica en un solo lugar, sino repartir las responsabilidades de forma clara y lógica entre cada una de las clases.

El aprendizaje más importante fue ver cómo las propias reglas y restricciones del programa son las que protegen la información. El encapsulamiento evitó que los datos de los pacientes se modificaran sin control, mientras que la sobrecarga de métodos nos permitió tener un controlador central en Sistema mucho más limpio y fácil de usar, sin necesidad de duplicar código. Por otro lado, manejar los límites de los arreglos estáticos con contadores manuales y usar copias auxiliares para ordenar la lista de pacientes nos probó que podemos procesar datos de forma eficiente sin arriesgar ni alterar los registros originales.

En conclusión, la etapa 1 cumplió al cien por ciento con todos los objetivos planteados al combinar el rigor del paradigma orientado a objetos con la lógica práctica que exigía el simulador, este proyecto no solo resolvió el problema inicial, sino que nos dejó un criterio de diseño mucho más claro para construir software modular, ordenado y listo para escalar en las siguientes entregas.

6. REFERENCIAS

- [1] Universidad Nacional Autónoma de México (UNAM CUAED), “Abstracción y Encapsulamiento,” Repositorio de la Coordinación de Universidad Abierta y Educación a Distancia. [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html
- [2] CiberAula, “Métodos de clase en Java,” Cursos de Programación en Java. [En línea]. Disponible en: https://www.ciberaula.com/cursos/java/metodos_clase.php
- [3] Oregoom, “Sobrecarga de Métodos y Constructores en Java,” Oregoom Tutoriales de Programación. [En línea]. Disponible en: <https://oregoom.com/java/sobrecarga-metodos-constructores/>
- [4] Tutoriales Programación Ya, “Patrón de Diseño State (Estado),” Diseño de Software: Patrones de Diseño GoF. [En línea]. Disponible en: <https://www.tutorialesprogramacionya.com/disenosoftware/patronesdisedisenogof/tema23.html>
- [5] Adictos al Trabajo, “Máquinas de Estado,” Autentia Real Business Solutions. [En línea]. Disponible en: <https://adictosaltrabajo.com/2011/08/12/state-machine/>
- [6] Aprender a Programar, “Arrays (arreglos) dinámicos y arrays estáticos: Definición, declaración y ejemplos en programación,” AprenderAProgramar.com. [En línea]. Disponible en: sitio web de AprenderAProgramar.com.
- [7] W3Schools, “Java String compareToIgnoreCase() Method,” W3Schools Online Web Tutorials. [En línea]. Disponible en: https://www.w3schools.com/java/ref_string_comparetoignorecase.asp